

# 06\_Analysis

July 11, 2026

## 1 Comparative Analysis of Dimension Reduction Methods

This notebook aggregates the results produced by the individual dimension reduction (DR) method notebooks — PCA, Kernel PCA (RBF and Polynomial), Gaussian Random Projection (GRP), and Sparse Random Projection (SRP) — into a single comparative analysis.

For each method, the following results are combined across all evaluated numbers of retained components (`n_components = [5, 10, 25, 50]`):

- Classification performance (Accuracy, Macro Precision, Macro Recall, Macro F1, Macro ROC AUC)
- Computational efficiency (fit time, transform time, process time, total runtime)
- Reconstruction error (MSE)

Representation quality is reported separately per method, since each method is evaluated using a measure specific to its own properties (explained variance for PCA, silhouette score for KPCA, pairwise distance correlation for GRP and SRP).

### 1.1 Workflow

1. Load the saved classification, reconstruction, and runtime summaries for each method.
2. Combine classification results into a single long-format table; compare methods across `n_components` for each classification metric.
3. Combine runtime results; compare fit time, transform time, and total runtime.
4. Combine reconstruction error results; compare with an explicit interpretive caveat.
5. Load and present representation quality results separately per method.
6. Identify best-performing configuration per method.
7. Save combined summary tables

### 1.2 1. Initial Configurations

```
[1]: # imports and paths
import warnings
warnings.filterwarnings("ignore")

import sys
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

project_root = Path("..").resolve()
sys.path.insert(0, str(project_root))

metrics_dir = Path("../results/metrics")
timings_dir = Path("../results/timings")
comparison_fig_dir = Path("../figures/Comparison")
comparison_fig_dir.mkdir(parents=True, exist_ok=True)

n_components_list = [5, 10, 25, 50]

```

[2]: *# Define Loader functions*

```

def load_classification_summary(path, method_name):
    df = pd.read_csv(path, index_col=[0, 1])
    df = df.reset_index()
    df.columns = ["n_components", "Metric", "Mean", "Std"]

    if (df["Metric"] == "Split").any():
        print(f"WARNING: 'Split' row found in {method_name} ({path.name}) - "
              f"removing it now, but verify the source notebook was actually_
↳re-run after the fix.")
        df = df[df["Metric"] != "Split"].reset_index(drop=True)

    df["Method"] = method_name
    return df

def load_reconstruction_summary(path, method_name):
    df = pd.read_csv(path, index_col=[0, 1])
    df = df.reset_index()
    df.columns = ["n_components", "Metric", "Mean", "Std"]
    df["Method"] = method_name
    return df

def load_runtime_summary(path, method_name):
    df = pd.read_csv(path, index_col=0)
    df.index.name = "n_components"
    df = df.reset_index()
    df["Method"] = method_name
    return df

```

### 1.3 2. Load and combine results

```
[3]: # Classification Results

classification_sources = {
    "PCA": metrics_dir / "PCA_classification_summary.csv",
    "GRP": metrics_dir / "grp_classification_summary.csv",
    "SRP": metrics_dir / "srp_classification_summary.csv",
    "KPCA (RBF)": metrics_dir / "KPCA_RBF_classification_summary.csv",
    "KPCA (Polynomial)": metrics_dir / "KPCA_Polynomial_classification_summary.
    ↪ csv",
}

classification_frames = []
for method_name, path in classification_sources.items():
    if not path.exists():
        print(f"MISSING FILE for {method_name}: {path}")
        continue
    classification_frames.append(load_classification_summary(path, method_name))

master_classification = pd.concat(classification_frames, ignore_index=True)
master_classification.to_csv(metrics_dir / "combined_classification_summary.
    ↪ csv", index=False)
print(master_classification)
```

|    | n_components |       | Metric    | Mean     | Std      | Method            |
|----|--------------|-------|-----------|----------|----------|-------------------|
| 0  | 5            |       | Accuracy  | 0.983762 | 0.010050 | PCA               |
| 1  | 5            | Macro | Precision | 0.981497 | 0.011348 | PCA               |
| 2  | 5            | Macro | Recall    | 0.981513 | 0.012935 | PCA               |
| 3  | 5            | Macro | F1        | 0.981118 | 0.012082 | PCA               |
| 4  | 5            | Macro | ROC AUC   | 0.999017 | 0.001243 | PCA               |
| .. | ...          |       | ...       | ...      | ...      | ...               |
| 95 | 50           |       | Accuracy  | 0.996502 | 0.005727 | KPCA (Polynomial) |
| 96 | 50           | Macro | Precision | 0.997428 | 0.004330 | KPCA (Polynomial) |
| 97 | 50           | Macro | Recall    | 0.994433 | 0.009540 | KPCA (Polynomial) |
| 98 | 50           | Macro | F1        | 0.995810 | 0.007156 | KPCA (Polynomial) |
| 99 | 50           | Macro | ROC AUC   | 0.999984 | 0.000040 | KPCA (Polynomial) |

[100 rows x 5 columns]

```
[4]: # Reconstruction Results

reconstruction_sources = {
    "PCA": metrics_dir / "PCA_reconstruction_summary.csv",
    "GRP": metrics_dir / "grp_reconstruction_summary.csv",
    "SRP": metrics_dir / "srp_reconstruction_summary.csv",
    "KPCA (RBF)": metrics_dir / "KPCA_RBF_reconstruction_summary.csv",
```

```

    "KPCA (Polynomial)": metrics_dir / "KPCA_Polynomial_reconstruction_summary.
↪csv",
}

reconstruction_frames = []
for method_name, path in reconstruction_sources.items():
    if not path.exists():
        print(f"MISSING FILE for {method_name}: {path}")
        continue
    reconstruction_frames.append(load_reconstruction_summary(path, method_name))

master_reconstruction = pd.concat(reconstruction_frames, ignore_index=True)
master_reconstruction.to_csv(metrics_dir / "combined_reconstruction_summary.
↪csv", index=False)
print(master_reconstruction)

```

|    | n_components |                | Metric | Mean     | Std      | Method            |
|----|--------------|----------------|--------|----------|----------|-------------------|
| 0  | 5            | Reconstruction | MSE    | 1.002959 | 0.027665 | PCA               |
| 1  | 10           | Reconstruction | MSE    | 0.840197 | 0.022255 | PCA               |
| 2  | 25           | Reconstruction | MSE    | 0.726573 | 0.019042 | PCA               |
| 3  | 50           | Reconstruction | MSE    | 0.661861 | 0.017601 | PCA               |
| 4  | 5            | Reconstruction | MSE    | 1.787700 | 0.025375 | GRP               |
| 5  | 10           | Reconstruction | MSE    | 1.787501 | 0.025394 | GRP               |
| 6  | 25           | Reconstruction | MSE    | 1.786514 | 0.025411 | GRP               |
| 7  | 50           | Reconstruction | MSE    | 1.783980 | 0.025427 | GRP               |
| 8  | 5            | Reconstruction | MSE    | 1.787953 | 0.025394 | SRP               |
| 9  | 10           | Reconstruction | MSE    | 1.787160 | 0.025367 | SRP               |
| 10 | 25           | Reconstruction | MSE    | 1.786252 | 0.025347 | SRP               |
| 11 | 50           | Reconstruction | MSE    | 1.784354 | 0.025366 | SRP               |
| 12 | 5            | Reconstruction | MSE    | 1.785459 | 0.025416 | KPCA (RBF)        |
| 13 | 10           | Reconstruction | MSE    | 1.785306 | 0.025416 | KPCA (RBF)        |
| 14 | 25           | Reconstruction | MSE    | 1.785259 | 0.025415 | KPCA (RBF)        |
| 15 | 50           | Reconstruction | MSE    | 1.785244 | 0.025415 | KPCA (RBF)        |
| 16 | 5            | Reconstruction | MSE    | 1.771192 | 0.025144 | KPCA (Polynomial) |
| 17 | 10           | Reconstruction | MSE    | 1.770245 | 0.025102 | KPCA (Polynomial) |
| 18 | 25           | Reconstruction | MSE    | 1.769964 | 0.025100 | KPCA (Polynomial) |
| 19 | 50           | Reconstruction | MSE    | 1.769877 | 0.025099 | KPCA (Polynomial) |

[5]: *# Runtime Results*

```

runtime_sources = {
    "PCA": timings_dir / "PCA_runtime_summary.csv",
    "GRP": timings_dir / "grp_runtime_summary.csv",
    "SRP": timings_dir / "srp_runtime_summary.csv",
    "KPCA (RBF)": timings_dir / "KPCA_RBF_runtime_summary.csv",
    "KPCA (Polynomial)": timings_dir / "KPCA_Polynomial_runtime_summary.csv",
}

```

```

runtime_frames = []
for method_name, path in runtime_sources.items():
    if not path.exists():
        print(f"MISSING FILE for {method_name}: {path}")
        continue
    runtime_frames.append(load_runtime_summary(path, method_name))

master_runtime = pd.concat(runtime_frames, ignore_index=True)
master_runtime.to_csv(timings_dir / "combined_runtime_summary.csv", index=False)
print(master_runtime)

```

|    | n_components | Fit Time Mean (s) | Fit Time Std (s) | \ |
|----|--------------|-------------------|------------------|---|
| 0  | 5            | 0.513217          | 0.022694         |   |
| 1  | 10           | 0.539847          | 0.022452         |   |
| 2  | 25           | 1.007973          | 0.101879         |   |
| 3  | 50           | 1.529165          | 0.189167         |   |
| 4  | 5            | 0.061567          | 0.013438         |   |
| 5  | 10           | 0.064921          | 0.004885         |   |
| 6  | 25           | 0.081648          | 0.005172         |   |
| 7  | 50           | 0.114419          | 0.005138         |   |
| 8  | 5            | 0.047886          | 0.005400         |   |
| 9  | 10           | 0.055698          | 0.007055         |   |
| 10 | 25           | 0.071158          | 0.011000         |   |
| 11 | 50           | 0.102967          | 0.014786         |   |
| 12 | 5            | 1.876627          | 0.151126         |   |
| 13 | 10           | 2.010755          | 0.165692         |   |
| 14 | 25           | 1.971899          | 0.131877         |   |
| 15 | 50           | 1.988653          | 0.111978         |   |
| 16 | 5            | 1.804201          | 0.143344         |   |
| 17 | 10           | 1.927246          | 0.125568         |   |
| 18 | 25           | 1.926556          | 0.113808         |   |
| 19 | 50           | 1.958444          | 0.132024         |   |

|    | Transform Time Mean (s) | Transform Time Std (s) | \ |
|----|-------------------------|------------------------|---|
| 0  | 0.012753                | 0.003922               |   |
| 1  | 0.013671                | 0.003900               |   |
| 2  | 0.014115                | 0.002538               |   |
| 3  | 0.017907                | 0.003537               |   |
| 4  | 0.010191                | 0.001340               |   |
| 5  | 0.010694                | 0.001080               |   |
| 6  | 0.013374                | 0.002303               |   |
| 7  | 0.015986                | 0.001262               |   |
| 8  | 0.005752                | 0.000758               |   |
| 9  | 0.006008                | 0.000716               |   |
| 10 | 0.006680                | 0.001898               |   |
| 11 | 0.007985                | 0.001906               |   |
| 12 | 0.173797                | 0.012890               |   |

|    |          |          |
|----|----------|----------|
| 13 | 0.172338 | 0.013174 |
| 14 | 0.165964 | 0.008909 |
| 15 | 0.168437 | 0.007890 |
| 16 | 0.124195 | 0.008563 |
| 17 | 0.124291 | 0.006669 |
| 18 | 0.124839 | 0.005914 |
| 19 | 0.124734 | 0.007475 |

|    | PCA Process Time Mean (s) | PCA Process Time Std (s) | Total Runtime (s) \ |
|----|---------------------------|--------------------------|---------------------|
| 0  | 0.525970                  | 0.023770                 | 100.288369          |
| 1  | 0.553517                  | 0.023876                 | 95.569041           |
| 2  | 1.022088                  | 0.101530                 | 105.091197          |
| 3  | 1.547072                  | 0.190449                 | 117.496555          |
| 4  | NaN                       | NaN                      | 82.174042           |
| 5  | NaN                       | NaN                      | 90.279018           |
| 6  | NaN                       | NaN                      | 113.364848          |
| 7  | NaN                       | NaN                      | 94.970422           |
| 8  | NaN                       | NaN                      | 84.311880           |
| 9  | NaN                       | NaN                      | 88.119596           |
| 10 | NaN                       | NaN                      | 106.245064          |
| 11 | NaN                       | NaN                      | 92.510952           |
| 12 | NaN                       | NaN                      | 133.244629          |
| 13 | NaN                       | NaN                      | 139.434955          |
| 14 | NaN                       | NaN                      | 135.000526          |
| 15 | NaN                       | NaN                      | 135.594122          |
| 16 | NaN                       | NaN                      | 129.086648          |
| 17 | NaN                       | NaN                      | 131.518091          |
| 18 | NaN                       | NaN                      | 132.176069          |
| 19 | NaN                       | NaN                      | 132.550743          |

|    | Method            | GRP Process Time Mean (s) | GRP Process Time Std (s) \ |
|----|-------------------|---------------------------|----------------------------|
| 0  | PCA               | NaN                       | NaN                        |
| 1  | PCA               | NaN                       | NaN                        |
| 2  | PCA               | NaN                       | NaN                        |
| 3  | PCA               | NaN                       | NaN                        |
| 4  | GRP               | 0.071758                  | 0.013543                   |
| 5  | GRP               | 0.075614                  | 0.005353                   |
| 6  | GRP               | 0.095022                  | 0.005563                   |
| 7  | GRP               | 0.130405                  | 0.005683                   |
| 8  | SRP               | NaN                       | NaN                        |
| 9  | SRP               | NaN                       | NaN                        |
| 10 | SRP               | NaN                       | NaN                        |
| 11 | SRP               | NaN                       | NaN                        |
| 12 | KPCA (RBF)        | NaN                       | NaN                        |
| 13 | KPCA (RBF)        | NaN                       | NaN                        |
| 14 | KPCA (RBF)        | NaN                       | NaN                        |
| 15 | KPCA (RBF)        | NaN                       | NaN                        |
| 16 | KPCA (Polynomial) | NaN                       | NaN                        |

|    |                   |     |     |
|----|-------------------|-----|-----|
| 17 | KPCA (Polynomial) | NaN | NaN |
| 18 | KPCA (Polynomial) | NaN | NaN |
| 19 | KPCA (Polynomial) | NaN | NaN |

|    | SRP Process Time Mean (s) | SRP Process Time Std (s) \ |
|----|---------------------------|----------------------------|
| 0  | NaN                       | NaN                        |
| 1  | NaN                       | NaN                        |
| 2  | NaN                       | NaN                        |
| 3  | NaN                       | NaN                        |
| 4  | NaN                       | NaN                        |
| 5  | NaN                       | NaN                        |
| 6  | NaN                       | NaN                        |
| 7  | NaN                       | NaN                        |
| 8  | 0.053638                  | 0.005858                   |
| 9  | 0.061705                  | 0.007451                   |
| 10 | 0.077838                  | 0.012239                   |
| 11 | 0.110953                  | 0.015965                   |
| 12 | NaN                       | NaN                        |
| 13 | NaN                       | NaN                        |
| 14 | NaN                       | NaN                        |
| 15 | NaN                       | NaN                        |
| 16 | NaN                       | NaN                        |
| 17 | NaN                       | NaN                        |
| 18 | NaN                       | NaN                        |
| 19 | NaN                       | NaN                        |

|    | KPCA Process Time Mean (s) | KPCA Process Time Std (s) |
|----|----------------------------|---------------------------|
| 0  | NaN                        | NaN                       |
| 1  | NaN                        | NaN                       |
| 2  | NaN                        | NaN                       |
| 3  | NaN                        | NaN                       |
| 4  | NaN                        | NaN                       |
| 5  | NaN                        | NaN                       |
| 6  | NaN                        | NaN                       |
| 7  | NaN                        | NaN                       |
| 8  | NaN                        | NaN                       |
| 9  | NaN                        | NaN                       |
| 10 | NaN                        | NaN                       |
| 11 | NaN                        | NaN                       |
| 12 | 2.050424                   | 0.157332                  |
| 13 | 2.183093                   | 0.167755                  |
| 14 | 2.137862                   | 0.130954                  |
| 15 | 2.157090                   | 0.110882                  |
| 16 | 1.928396                   | 0.146462                  |
| 17 | 2.051537                   | 0.125363                  |
| 18 | 2.051395                   | 0.114906                  |
| 19 | 2.083178                   | 0.131091                  |

### 1.4 3. Classification Performance Comparison

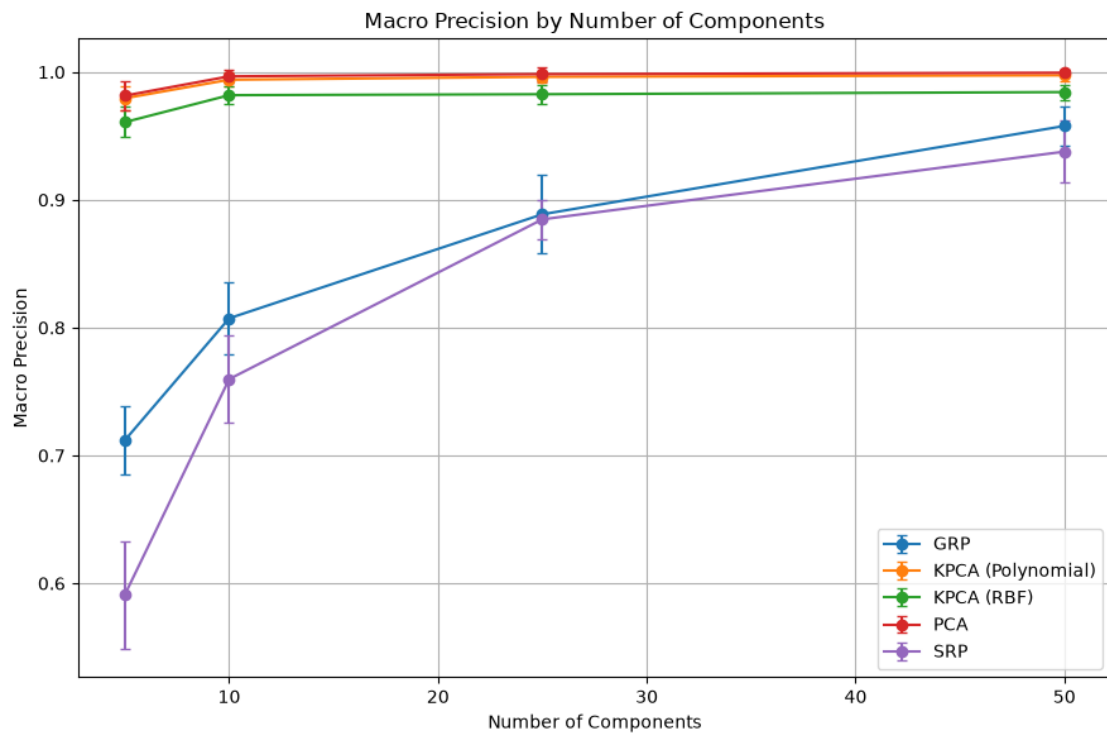
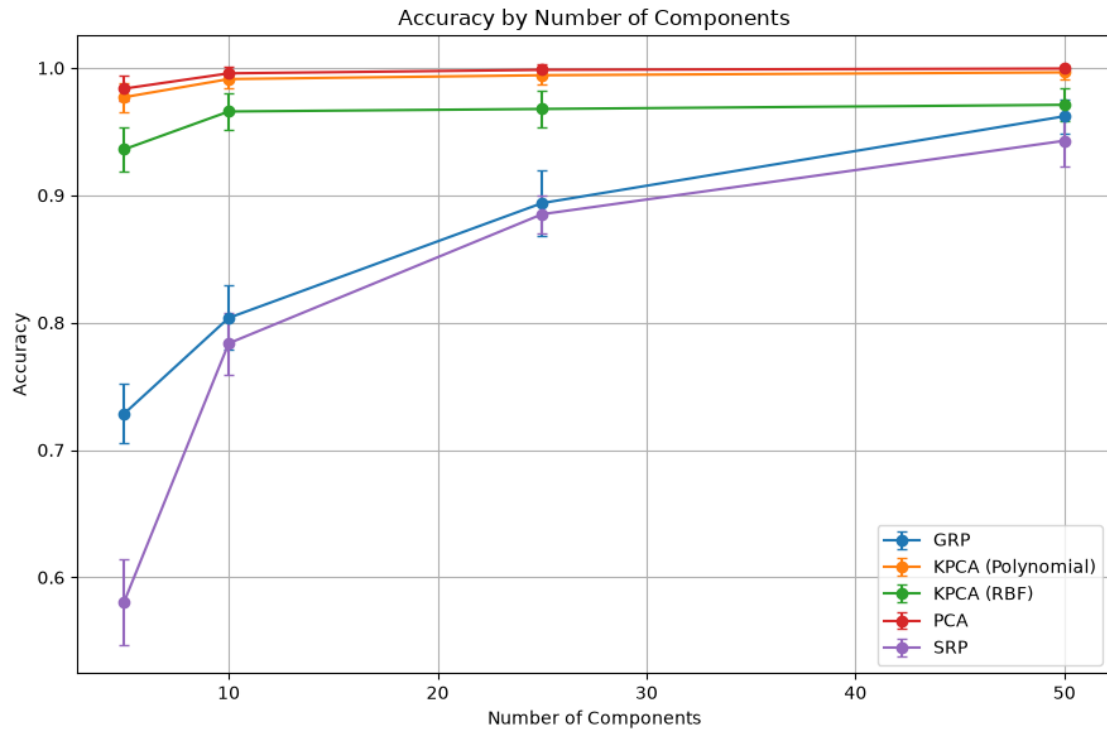
The following plots compare each classification metric across all five DR configurations (PCA, GRP, SRP, KPCA-RBF, KPCA-Polynomial) as a function of the number of retained components. Error bars represent one standard deviation across the 25 cross-validation splits.

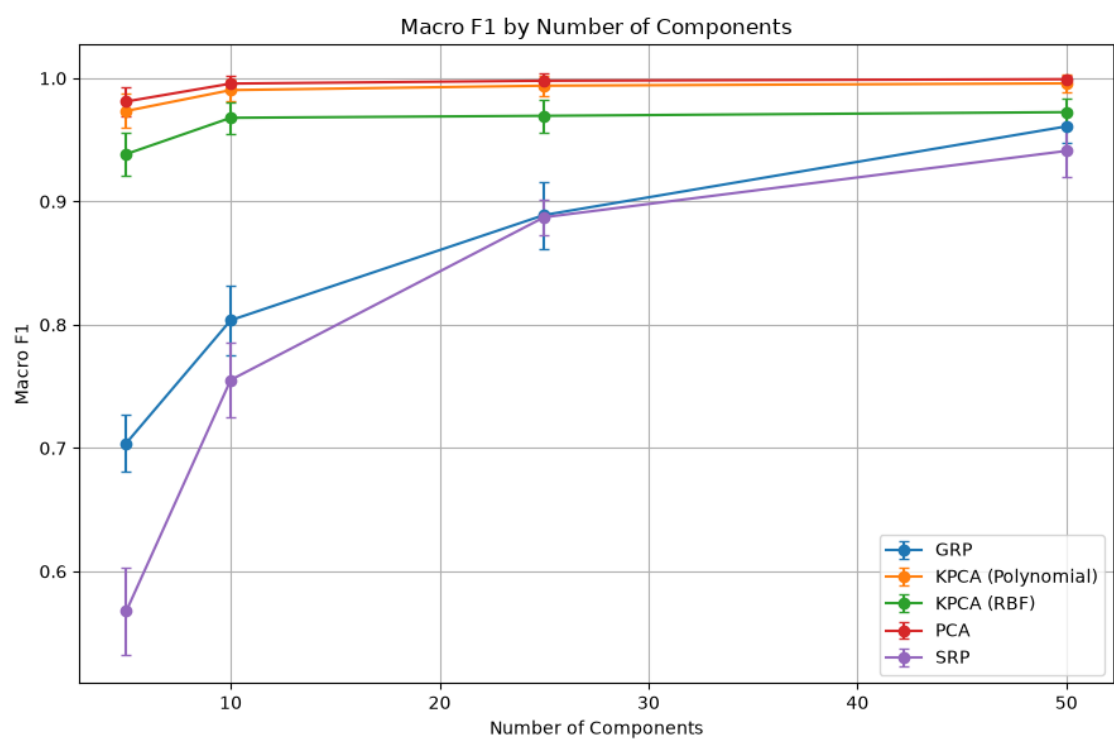
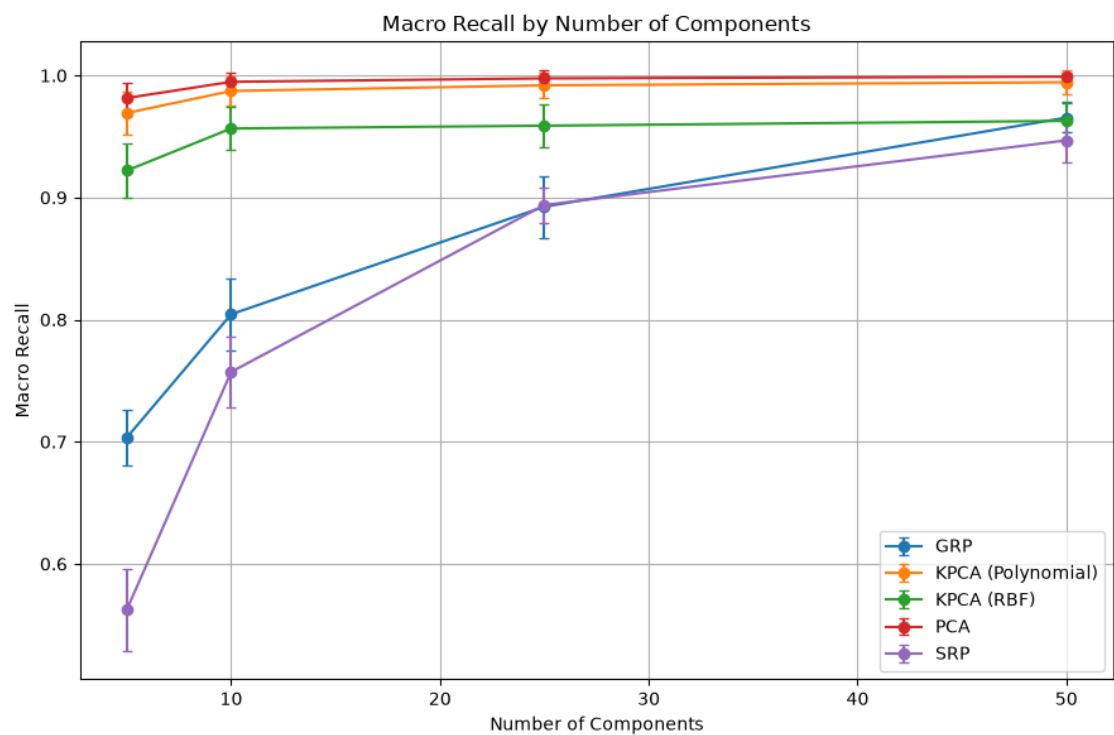
```
[6]: # Plotting function + generate all metric comparison plot

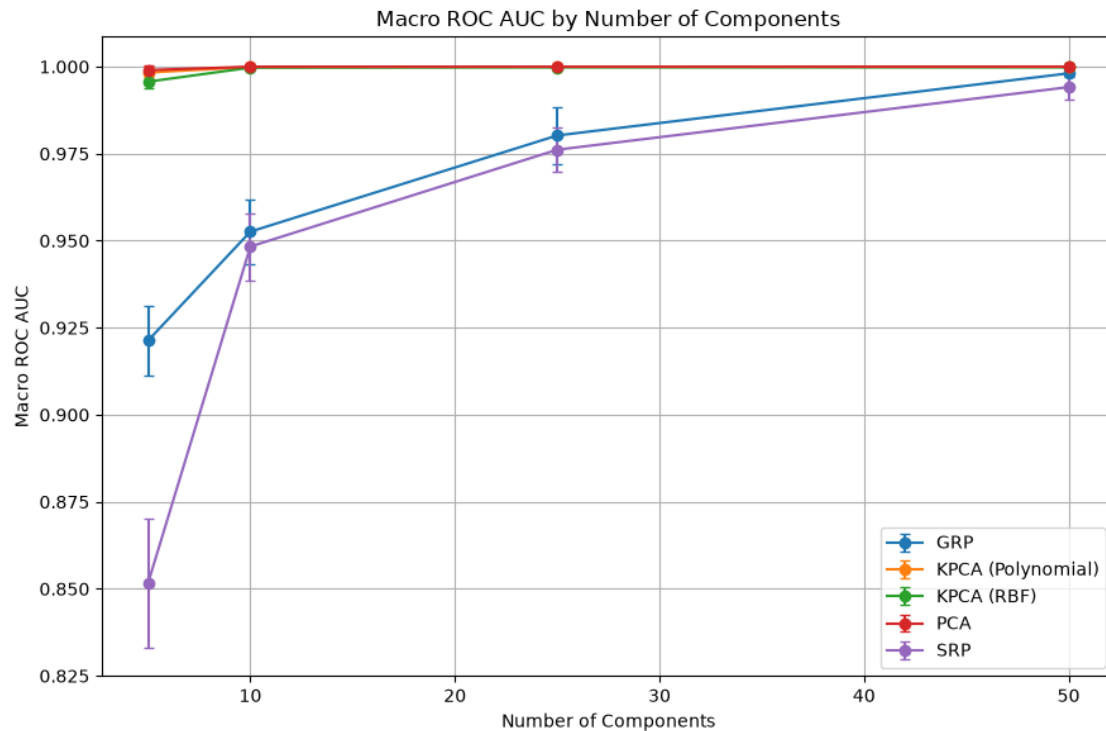
def plot_metric_comparison(master_df, metric_name, save_path=None):
    plt.figure(figsize=(9, 6))
    subset = master_df[master_df["Metric"] == metric_name]
    for method_name, group in subset.groupby("Method"):
        group = group.sort_values("n_components")
        plt.errorbar(
            group["n_components"], group["Mean"], yerr=group["Std"],
            marker="o", capsize=3, label=method_name,
        )
    plt.xlabel("Number of Components")
    plt.ylabel(metric_name)
    plt.title(f"{metric_name} by Number of Components")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.show()

for metric in ["Accuracy", "Macro Precision", "Macro Recall", "Macro F1", "↵
↵Macro ROC AUC"]:
    fname = metric.replace(" ", "_").lower()
    plot_metric_comparison(
        master_classification, metric,
        save_path=comparison_fig_dir / f"comparison_{fname}.png",
    )
```









```
[7]: # Wide-format pivot tables for every classification metric
classification_metrics = ["Accuracy", "Macro Precision", "Macro Recall", "Macro_
↳F1", "Macro ROC AUC"]

pivot_tables = {}
for metric in classification_metrics:
    pivot = (
        master_classification[master_classification["Metric"] == metric]
        .pivot(index="n_components", columns="Method", values="Mean")
        .round(4)
    )
    pivot_tables[metric] = pivot

    fname = metric.replace(" ", "_").lower()
    pivot.to_csv(metrics_dir / f"comparison_{fname}_pivot.csv")

# Display all five
for metric, pivot in pivot_tables.items():
    print(f"\n{metric}")
    print(pivot)
```

| Accuracy |     |                   |            |     |     |
|----------|-----|-------------------|------------|-----|-----|
| Method   | GRP | KPCA (Polynomial) | KPCA (RBF) | PCA | SRP |

| n_components |        |        |        |        |        |
|--------------|--------|--------|--------|--------|--------|
| 5            | 0.7286 | 0.9770 | 0.9361 | 0.9838 | 0.5810 |
| 10           | 0.8037 | 0.9913 | 0.9658 | 0.9958 | 0.7838 |
| 25           | 0.8939 | 0.9943 | 0.9678 | 0.9985 | 0.8851 |
| 50           | 0.9620 | 0.9965 | 0.9710 | 0.9995 | 0.9428 |

#### Macro Precision

| Method       | GRP    | KPCA (Polynomial) | KPCA (RBF) | PCA    | SRP    |
|--------------|--------|-------------------|------------|--------|--------|
| n_components |        |                   |            |        |        |
| 5            | 0.7119 | 0.9795            | 0.9608     | 0.9815 | 0.5910 |
| 10           | 0.8073 | 0.9940            | 0.9819     | 0.9966 | 0.7597 |
| 25           | 0.8887 | 0.9962            | 0.9826     | 0.9984 | 0.8848 |
| 50           | 0.9578 | 0.9974            | 0.9843     | 0.9994 | 0.9377 |

#### Macro Recall

| Method       | GRP    | KPCA (Polynomial) | KPCA (RBF) | PCA    | SRP    |
|--------------|--------|-------------------|------------|--------|--------|
| n_components |        |                   |            |        |        |
| 5            | 0.7034 | 0.9692            | 0.9221     | 0.9815 | 0.5624 |
| 10           | 0.8044 | 0.9873            | 0.9566     | 0.9948 | 0.7572 |
| 25           | 0.8925 | 0.9919            | 0.9589     | 0.9976 | 0.8939 |
| 50           | 0.9655 | 0.9944            | 0.9628     | 0.9990 | 0.9468 |

#### Macro F1

| Method       | GRP    | KPCA (Polynomial) | KPCA (RBF) | PCA    | SRP    |
|--------------|--------|-------------------|------------|--------|--------|
| n_components |        |                   |            |        |        |
| 5            | 0.7039 | 0.9734            | 0.9385     | 0.9811 | 0.5677 |
| 10           | 0.8037 | 0.9904            | 0.9679     | 0.9957 | 0.7552 |
| 25           | 0.8890 | 0.9939            | 0.9695     | 0.9980 | 0.8872 |
| 50           | 0.9609 | 0.9958            | 0.9725     | 0.9992 | 0.9410 |

#### Macro ROC AUC

| Method       | GRP    | KPCA (Polynomial) | KPCA (RBF) | PCA   | SRP    |
|--------------|--------|-------------------|------------|-------|--------|
| n_components |        |                   |            |       |        |
| 5            | 0.9213 | 0.9983            | 0.9957     | 0.999 | 0.8517 |
| 10           | 0.9526 | 0.9999            | 0.9997     | 1.000 | 0.9483 |
| 25           | 0.9802 | 1.0000            | 0.9998     | 1.000 | 0.9761 |
| 50           | 0.9981 | 1.0000            | 0.9998     | 1.000 | 0.9941 |

```
[8]: # Combined Mean ± Std pivot, formatted as a single string per cell
combined_pivots = {}
for metric in classification_metrics:
    subset = master_classification[master_classification["Metric"] == metric].
    ↪copy()
    subset["Formatted"] = subset["Mean"].round(4).astype(str) + " ± " +
    ↪subset["Std"].round(4).astype(str)
    pivot = subset.pivot(index="n_components", columns="Method",
    ↪values="Formatted")
```

```

combined_pivots[metric] = pivot

fname = metric.replace(" ", "_").lower()
pivot.to_csv(metrics_dir / f"comparison_{fname}_pivot_with_std.csv")

for metric, pivot in combined_pivots.items():
    print(f"\n{metric} (Mean ± Std)")
    print(pivot)

```

#### Accuracy (Mean ± Std)

| Method       | GRP             | KPCA (Polynomial) | KPCA (RBF) \    |
|--------------|-----------------|-------------------|-----------------|
| n_components |                 |                   |                 |
| 5            | 0.7286 ± 0.0231 | 0.977 ± 0.0114    | 0.9361 ± 0.0176 |
| 10           | 0.8037 ± 0.0252 | 0.9913 ± 0.0072   | 0.9658 ± 0.0142 |
| 25           | 0.8939 ± 0.026  | 0.9943 ± 0.0067   | 0.9678 ± 0.0146 |
| 50           | 0.962 ± 0.0132  | 0.9965 ± 0.0057   | 0.971 ± 0.0126  |

| Method       | PCA             | SRP             |
|--------------|-----------------|-----------------|
| n_components |                 |                 |
| 5            | 0.9838 ± 0.0101 | 0.581 ± 0.0337  |
| 10           | 0.9958 ± 0.0056 | 0.7838 ± 0.0243 |
| 25           | 0.9985 ± 0.0041 | 0.8851 ± 0.0152 |
| 50           | 0.9995 ± 0.0017 | 0.9428 ± 0.0198 |

#### Macro Precision (Mean ± Std)

| Method       | GRP             | KPCA (Polynomial) | KPCA (RBF) \    |
|--------------|-----------------|-------------------|-----------------|
| n_components |                 |                   |                 |
| 5            | 0.7119 ± 0.027  | 0.9795 ± 0.0093   | 0.9608 ± 0.0118 |
| 10           | 0.8073 ± 0.0282 | 0.994 ± 0.0054    | 0.9819 ± 0.0066 |
| 25           | 0.8887 ± 0.0307 | 0.9962 ± 0.0048   | 0.9826 ± 0.0072 |
| 50           | 0.9578 ± 0.0152 | 0.9974 ± 0.0043   | 0.9843 ± 0.006  |

| Method       | PCA             | SRP             |
|--------------|-----------------|-----------------|
| n_components |                 |                 |
| 5            | 0.9815 ± 0.0113 | 0.591 ± 0.0419  |
| 10           | 0.9966 ± 0.0048 | 0.7597 ± 0.0339 |
| 25           | 0.9984 ± 0.0051 | 0.8848 ± 0.0155 |
| 50           | 0.9994 ± 0.0019 | 0.9377 ± 0.0242 |

#### Macro Recall (Mean ± Std)

| Method       | GRP             | KPCA (Polynomial) | KPCA (RBF) \    |
|--------------|-----------------|-------------------|-----------------|
| n_components |                 |                   |                 |
| 5            | 0.7034 ± 0.0232 | 0.9692 ± 0.0172   | 0.9221 ± 0.0222 |
| 10           | 0.8044 ± 0.0298 | 0.9873 ± 0.0116   | 0.9566 ± 0.0176 |
| 25           | 0.8925 ± 0.0254 | 0.9919 ± 0.0107   | 0.9589 ± 0.0178 |
| 50           | 0.9655 ± 0.0117 | 0.9944 ± 0.0095   | 0.9628 ± 0.0157 |

| Method       | PCA             | SRP             |
|--------------|-----------------|-----------------|
| n_components |                 |                 |
| 5            | 0.9815 ± 0.0129 | 0.5624 ± 0.0333 |
| 10           | 0.9948 ± 0.0076 | 0.7572 ± 0.0286 |
| 25           | 0.9976 ± 0.0064 | 0.8939 ± 0.0146 |
| 50           | 0.999 ± 0.0035  | 0.9468 ± 0.0183 |

Macro F1 (Mean ± Std)

| Method       | GRP KPCA (Polynomial) | KPCA (RBF) \    |
|--------------|-----------------------|-----------------|
| n_components |                       |                 |
| 5            | 0.7039 ± 0.0227       | 0.9734 ± 0.014  |
| 10           | 0.8037 ± 0.0283       | 0.9904 ± 0.0089 |
| 25           | 0.889 ± 0.0272        | 0.9939 ± 0.008  |
| 50           | 0.9609 ± 0.0133       | 0.9958 ± 0.0072 |

| Method       | PCA             | SRP             |
|--------------|-----------------|-----------------|
| n_components |                 |                 |
| 5            | 0.9811 ± 0.0121 | 0.5677 ± 0.035  |
| 10           | 0.9957 ± 0.0061 | 0.7552 ± 0.03   |
| 25           | 0.998 ± 0.0057  | 0.8872 ± 0.0143 |
| 50           | 0.9992 ± 0.0028 | 0.941 ± 0.0215  |

Macro ROC AUC (Mean ± Std)

| Method       | GRP KPCA (Polynomial) | KPCA (RBF) \    |
|--------------|-----------------------|-----------------|
| n_components |                       |                 |
| 5            | 0.9213 ± 0.0101       | 0.9983 ± 0.0016 |
| 10           | 0.9526 ± 0.0094       | 0.9999 ± 0.0001 |
| 25           | 0.9802 ± 0.0083       | 1.0 ± 0.0001    |
| 50           | 0.9981 ± 0.0011       | 1.0 ± 0.0       |

| Method       | PCA            | SRP             |
|--------------|----------------|-----------------|
| n_components |                |                 |
| 5            | 0.999 ± 0.0012 | 0.8517 ± 0.0185 |
| 10           | 1.0 ± 0.0001   | 0.9483 ± 0.0096 |
| 25           | 1.0 ± 0.0      | 0.9761 ± 0.0063 |
| 50           | 1.0 ± 0.0      | 0.9941 ± 0.0034 |

## 1.5 4. Computational Efficiency Comparison

Fit time, transform time, and total runtime are compared across methods. Fit time is shown on a log scale given the substantial difference in cost between KPCA (kernel matrix computation) and the linear/random-projection methods.

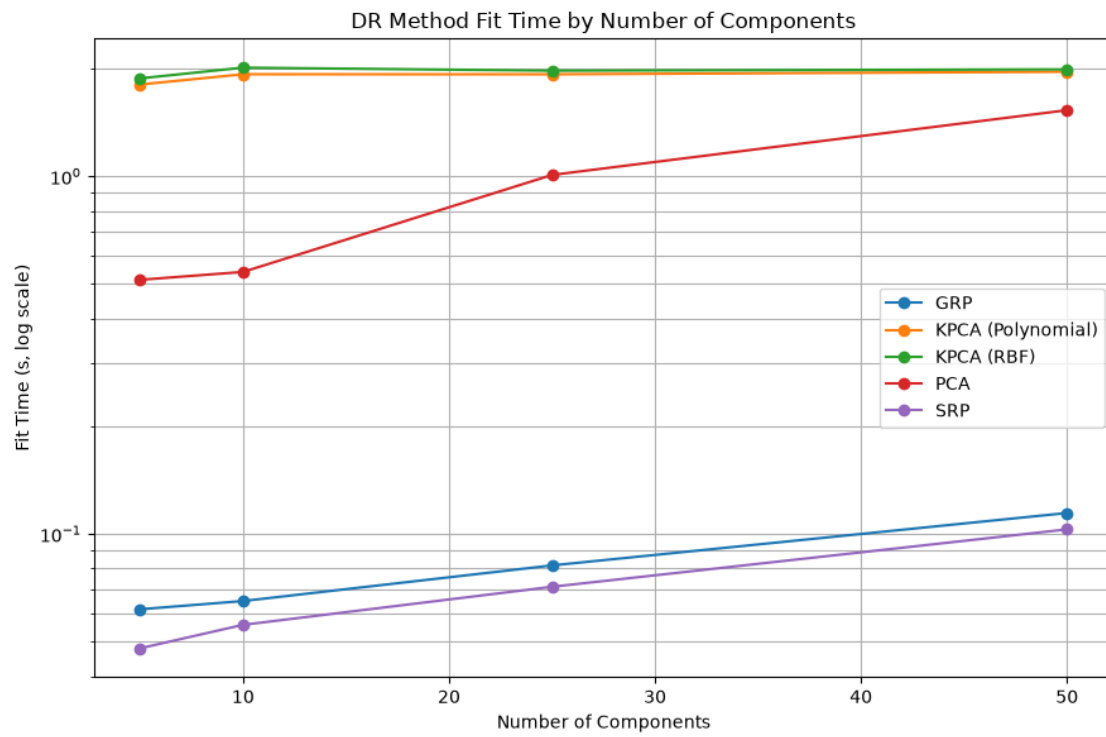
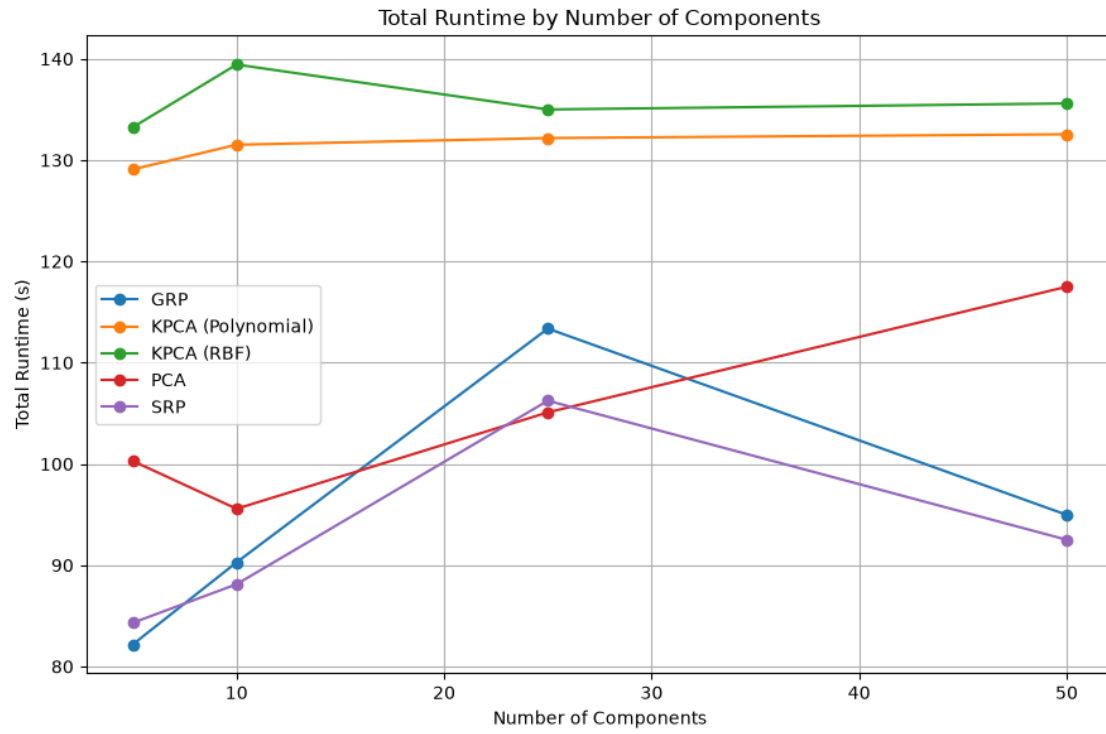
```
[9]: # Total runtime
plt.figure(figsize=(9, 6))
for method_name, group in master_runtime.groupby("Method"):
    group = group.sort_values("n_components")
```

```

    plt.plot(group["n_components"], group["Total Runtime (s)"], marker="o",
        ↳label=method_name)
plt.xlabel("Number of Components")
plt.ylabel("Total Runtime (s)")
plt.title("Total Runtime by Number of Components")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(comparison_fig_dir / "comparison_total_runtime.png", dpi=300,
    ↳bbox_inches="tight")
plt.show()

# Fit time (log scale)
plt.figure(figsize=(9, 6))
for method_name, group in master_runtime.groupby("Method"):
    group = group.sort_values("n_components")
    plt.plot(group["n_components"], group["Fit Time Mean (s)"], marker="o",
        ↳label=method_name)
plt.xlabel("Number of Components")
plt.ylabel("Fit Time (s, log scale)")
plt.yscale("log")
plt.title("DR Method Fit Time by Number of Components")
plt.legend()
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig(comparison_fig_dir / "comparison_fit_time.png", dpi=300,
    ↳bbox_inches="tight")
plt.show()

```



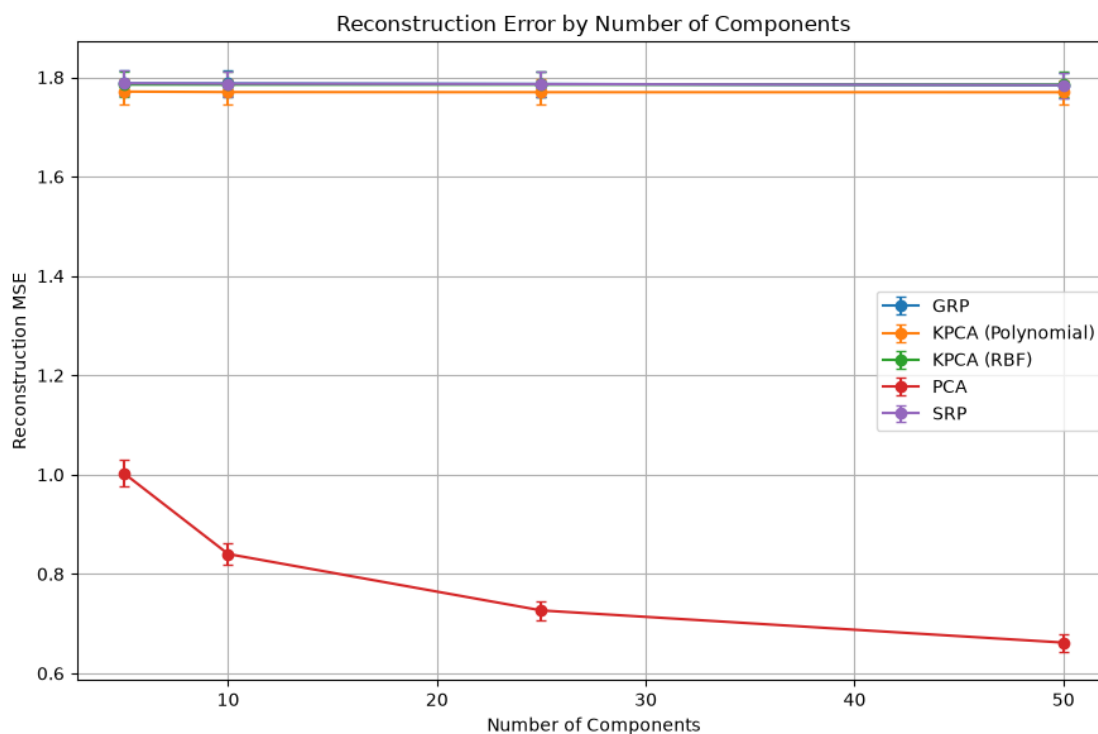


## 1.6 5. Reconstruction Error Comparison

**Note:** reconstruction is computed differently across methods: PCA uses its exact inverse transform, KPCA uses the approximate pre-image method, and GRP/SRP use the Moore-Penrose pseudo-inverse of the projection matrix. As stated in Chapter 4, these values are therefore interpreted within the context of each method rather than as directly equivalent measures, even though they are plotted together here for visual comparison.

```
[10]: # Reconstruction Plot

plt.figure(figsize=(9, 6))
for method_name, group in master_reconstruction.groupby("Method"):
    group = group.sort_values("n_components")
    plt.errorbar(
        group["n_components"], group["Mean"], yerr=group["Std"],
        marker="o", capsize=3, label=method_name,
    )
plt.xlabel("Number of Components")
plt.ylabel("Reconstruction MSE")
plt.title("Reconstruction Error by Number of Components")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(comparison_fig_dir / "comparison_reconstruction_mse.png", dpi=300,
            bbox_inches="tight")
plt.show()
```



## 1.7 6. Representation Quality (Reported Separately per Method)

Consistent with Chapter 4, representation quality is reported descriptively per method rather than combined into a single plot, since each measure captures a different property of the reduced feature space. The exception is GRP and SRP, which are compared directly since both are evaluated using the same measure (pairwise distance correlation).

```
[11]: # PCA: explained variance at reported component counts
```

```
pca_var_df = pd.read_csv(metrics_dir / "PCA_explained_variance.csv")
pca_repr_summary = pca_var_df[
    pca_var_df["Principal Component"].isin(n_components_list)
][["Principal Component", "Cumulative Explained Variance"]].rename(
    columns={"Principal Component": "n_components"}
)
pca_repr_summary
```

```
[11]:
```

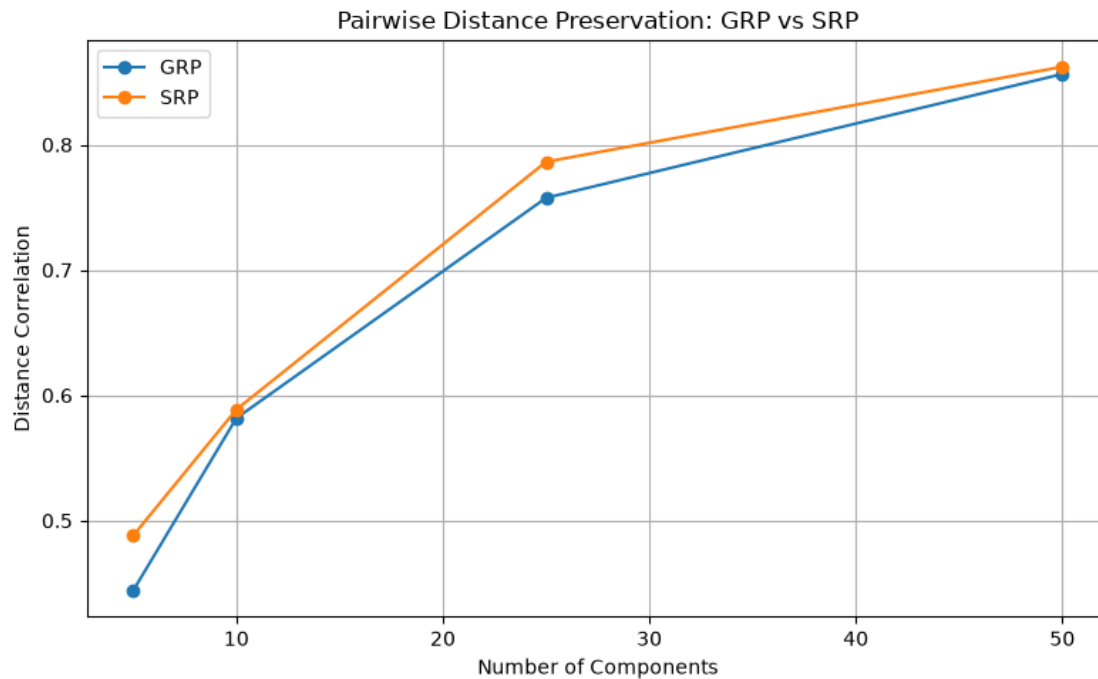
|    | n_components | Cumulative Explained Variance |
|----|--------------|-------------------------------|
| 4  | 5            | 0.362987                      |
| 9  | 10           | 0.465855                      |
| 24 | 25           | 0.559722                      |
| 49 | 50           | 0.636982                      |

```
[12]: # GRP vs SRP: distance preservation (shared metric, safe to combine)
```

```
grp_dist = pd.read_csv(metrics_dir / "grp_distance_preservation.csv")
grp_dist["Method"] = "GRP"
srp_dist = pd.read_csv(metrics_dir / "srp_distance_preservation.csv")
srp_dist["Method"] = "SRP"
distance_df = pd.concat([grp_dist, srp_dist], ignore_index=True)

plt.figure(figsize=(8, 5))
for method_name, group in distance_df.groupby("Method"):
    group = group.sort_values("n_components")
    plt.plot(group["n_components"], group["Distance Correlation"], marker="o",
             label=method_name)
plt.xlabel("Number of Components")
plt.ylabel("Distance Correlation")
plt.title("Pairwise Distance Preservation: GRP vs SRP")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(comparison_fig_dir / "comparison_distance_preservation_grp_srp.
            png", dpi=300, bbox_inches="tight")
plt.show()
```

```
distance_df.pivot(index="n_components", columns="Method", values="Distance_
↳Correlation")
```



```
[12]: Method          GRP          SRP
n_components
5          0.444087  0.487961
10         0.581715  0.588652
25         0.757538  0.786298
50         0.856329  0.862146
```

```
[13]: ## KPCA: silhouette by components

kpca_silhouette_df = pd.read_csv(metrics_dir / "KPCA_silhouette_by_components.
↳csv")
kpca_silhouette_df.pivot(index="n_components", columns="Kernel",
↳values="Silhouette Score")
```

```
[13]: Kernel          Polynomial          RBF
n_components
5          0.404880  0.379140
10         0.342880  0.312136
25         0.270812  0.223151
50         0.229727  0.182150
```

## 1.8 7. Best configuration per method

```
[14]: best_configs = (  
    master_classification[master_classification["Metric"] == "Accuracy"]  
    .sort_values("Mean", ascending=False)  
    .groupby("Method")  
    .first()  
    .reset_index()[["Method", "n_components", "Mean", "Std"]]  
    .rename(columns={"Mean": "Best Accuracy", "Std": "Accuracy Std"})  
    .sort_values("Best Accuracy", ascending=False)  
)  
best_configs.to_csv(metrics_dir / "best_configuration_per_method.csv",  
    ↪index=False)  
best_configs
```

```
[14]:
```

|   | Method            | n_components | Best Accuracy | Accuracy Std |
|---|-------------------|--------------|---------------|--------------|
| 3 | PCA               | 50           | 0.999500      | 0.001731     |
| 1 | KPCA (Polynomial) | 50           | 0.996502      | 0.005727     |
| 2 | KPCA (RBF)        | 50           | 0.971028      | 0.012622     |
| 0 | GRP               | 50           | 0.962042      | 0.013248     |
| 4 | SRP               | 50           | 0.942823      | 0.019804     |

```
[ ]:
```